Advanced Data Science
ML with Time-series

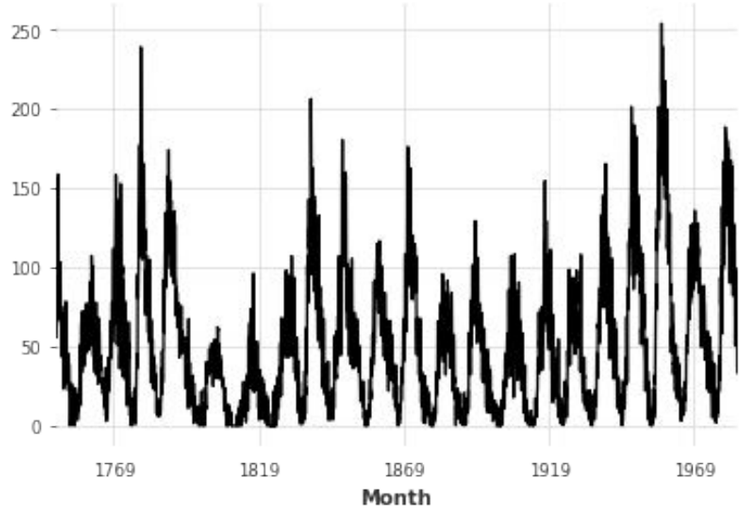
Thomas Moreau
Mathurin Massias

What are the different types of time-series?

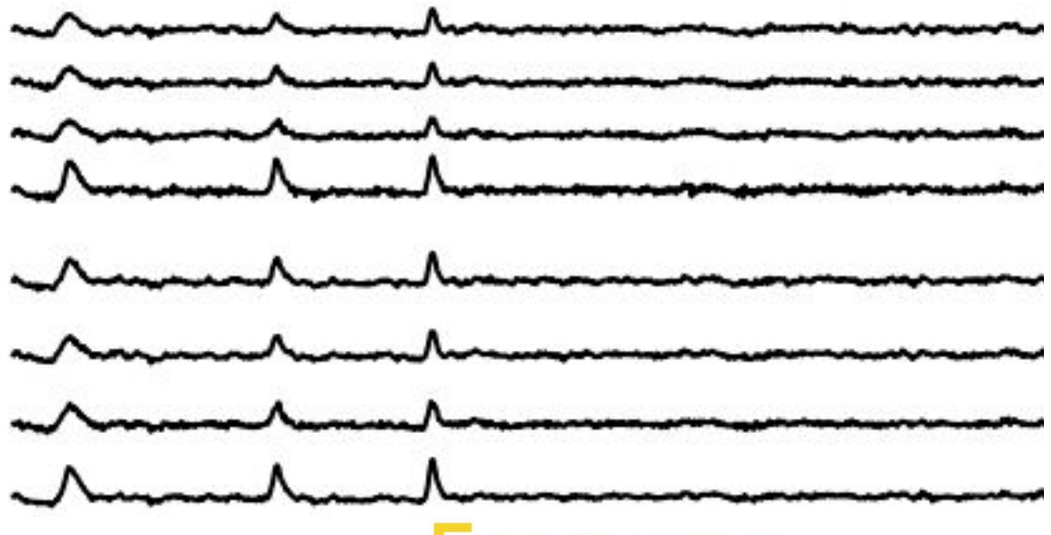
Can you give examples of cases where you would need to process signals with your applications?

Univariate vs Multivariate time-series

Sun spots data



EEG data



$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{t-1} \\ x_t \end{bmatrix}$$

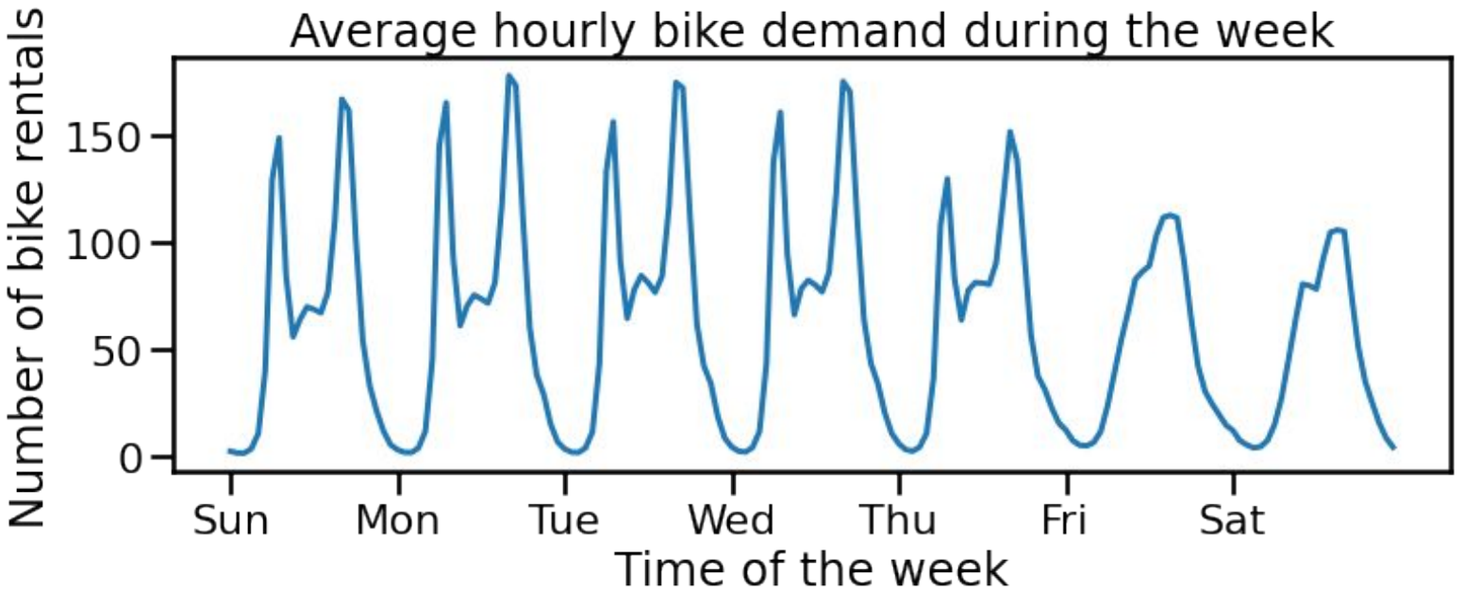
Time

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \dots & x_1^p \\ x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \dots & \dots & \vdots \\ x_{t-1}^1 & x_{t-1}^2 & \dots & x_{t-1}^p \\ x_t^1 & x_t^2 & \dots & x_t^p \end{bmatrix}$$

Features

Time

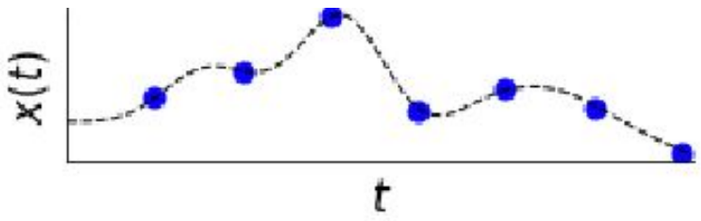
Multivariate time-series with covariates



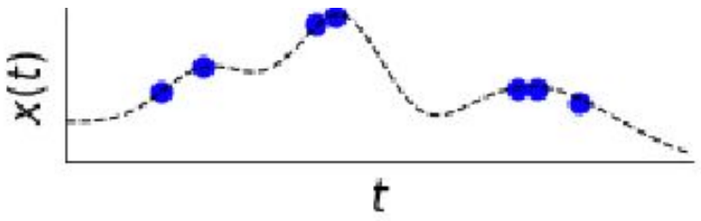
$$X = \text{concat}\left(\underbrace{\begin{bmatrix} x_1^1 & x_1^2 & \dots & x_1^p \\ x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \dots & \dots & \vdots \\ x_t^1 & x_t^2 & \dots & x_t^p \end{bmatrix}}_{\text{time-series}}, \underbrace{\begin{bmatrix} z_1^1 & \dots & z_1^q \\ z_2^1 & \dots & z_2^q \\ \vdots & \dots & \vdots \\ z_t^1 & \dots & z_t^q \end{bmatrix}}_{\text{covariates}} \right)$$

(e.g. holidays, weather etc.)

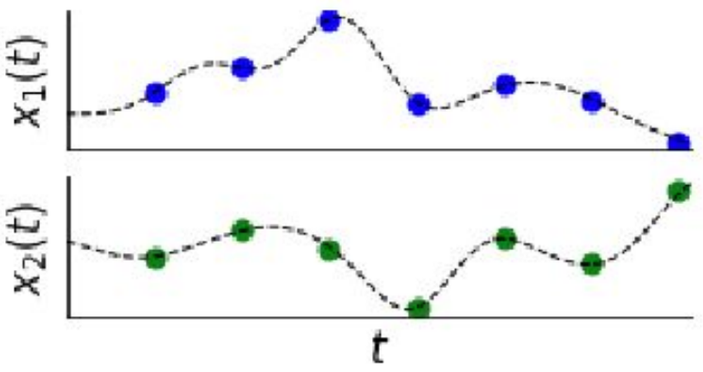
Regularly vs Irregularly sampled TS



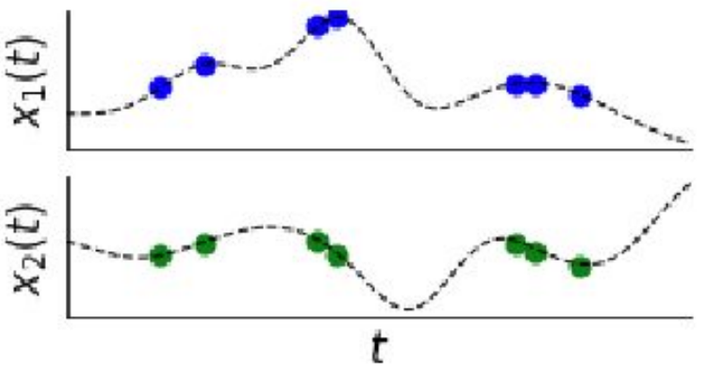
Univariate regularly sampled



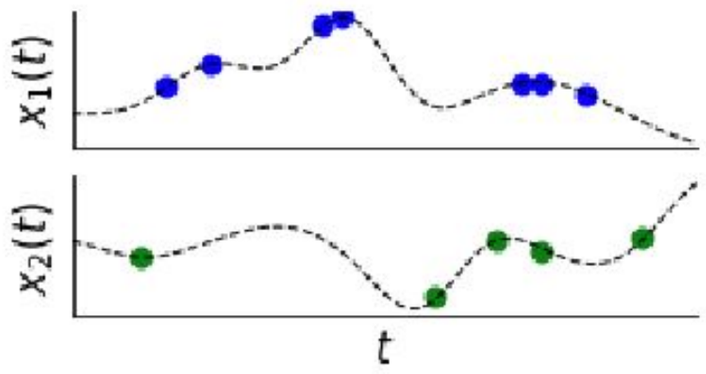
Univarite irregulalry sampled



Multivariate regularly sampled



Multivariate irregularly sampled (aligned)

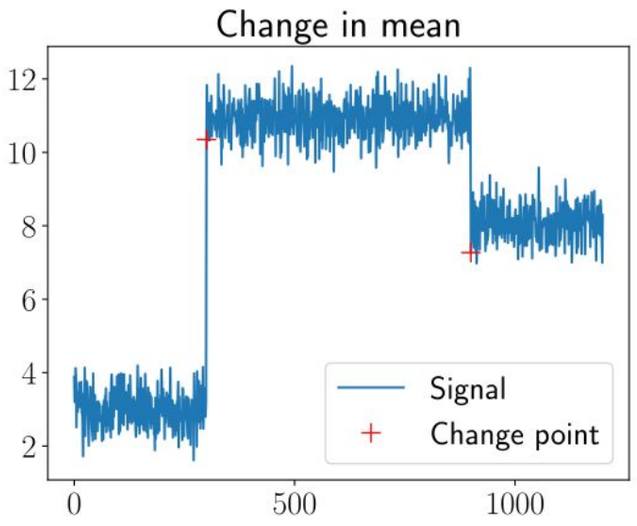


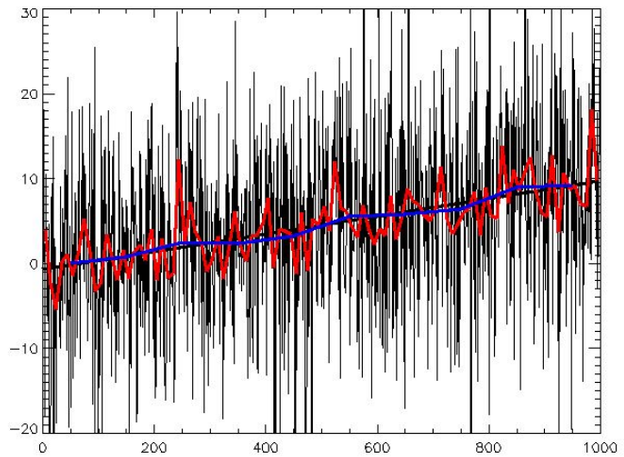
Multivariate irregularly sampled (unaligned)

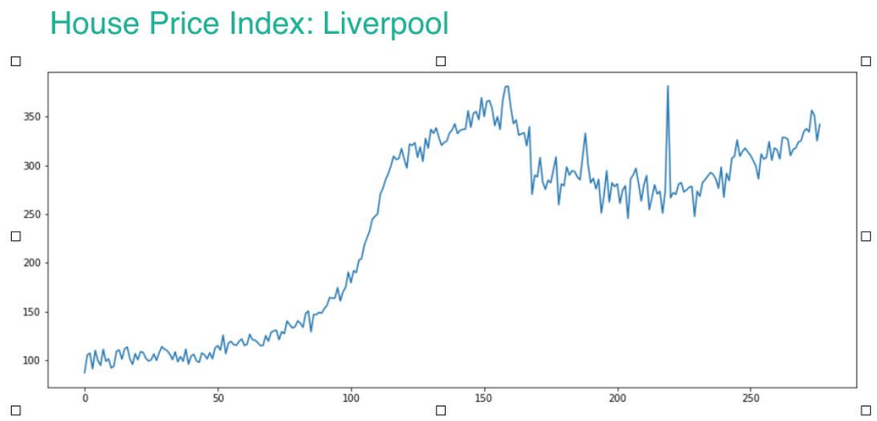
- Regular grids can also have missing values.
- Sometimes, irregular TS can be resampled (signals)

Looking at time series (and some jargon)

Smoothness, noise and stationarity







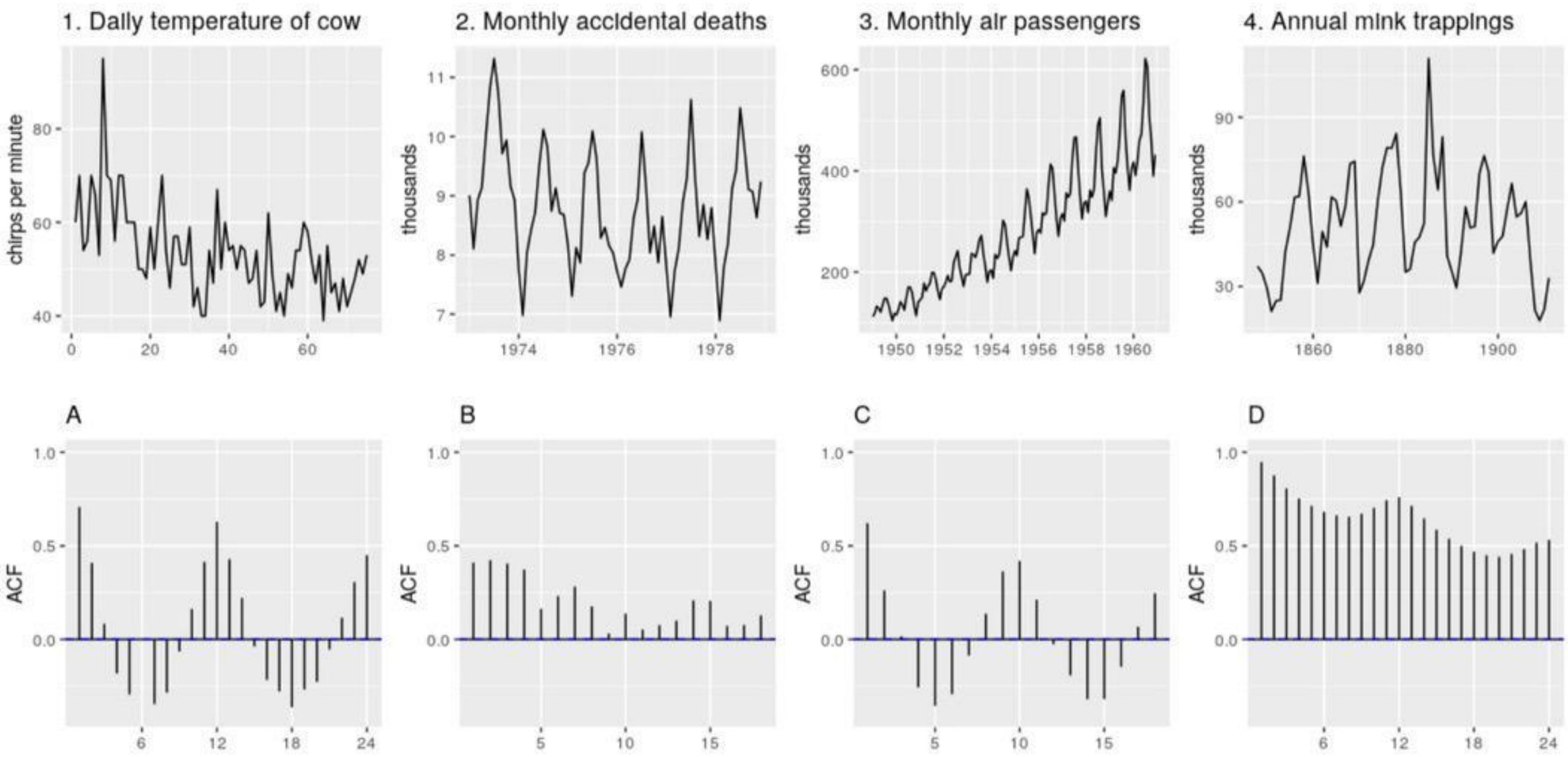
- Smoothness measure how close two samples can be.
- Noise can hide the overall pattern
- Non-stationarity are points where the statistics or the dynamic of the signal change

Autocorrelation

- Measures the correlation between x_t and x_{t-k}

$$r_k = \frac{\sum_{t=k+1}^{T} (x_t - \bar{x})(x_{t-k} - \bar{x})}{\sum_{t=1}^{T} (x_t - \bar{x})^2}$$

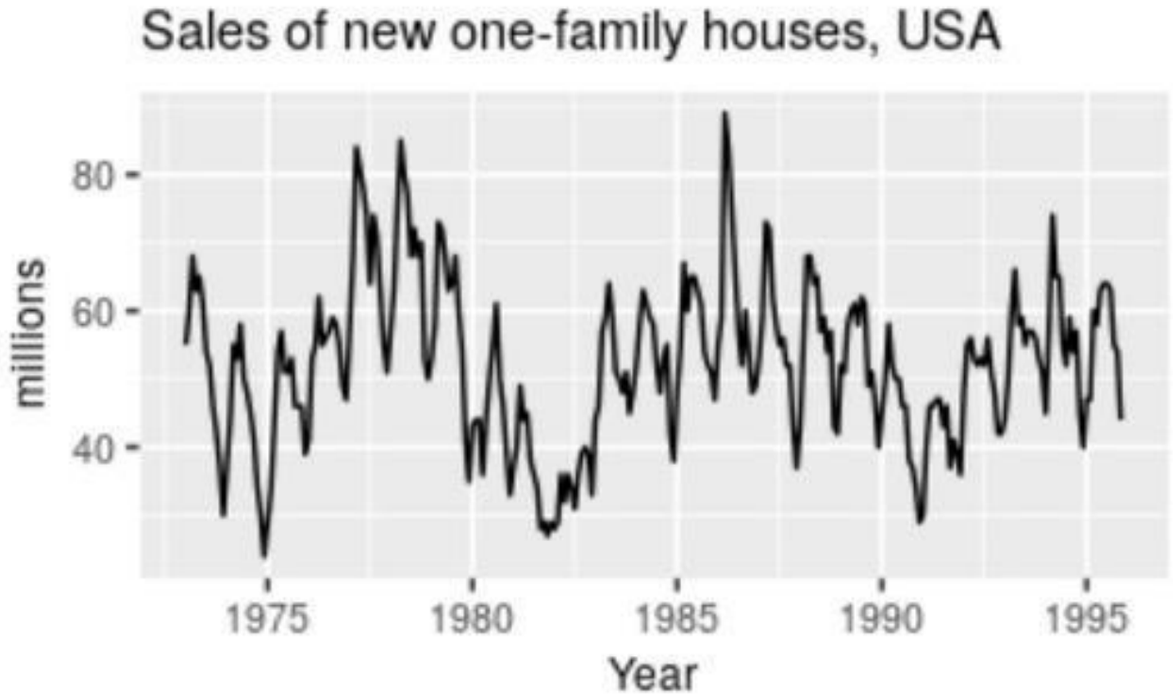
- Can you match the autocorrelation function (ACF) to the data?

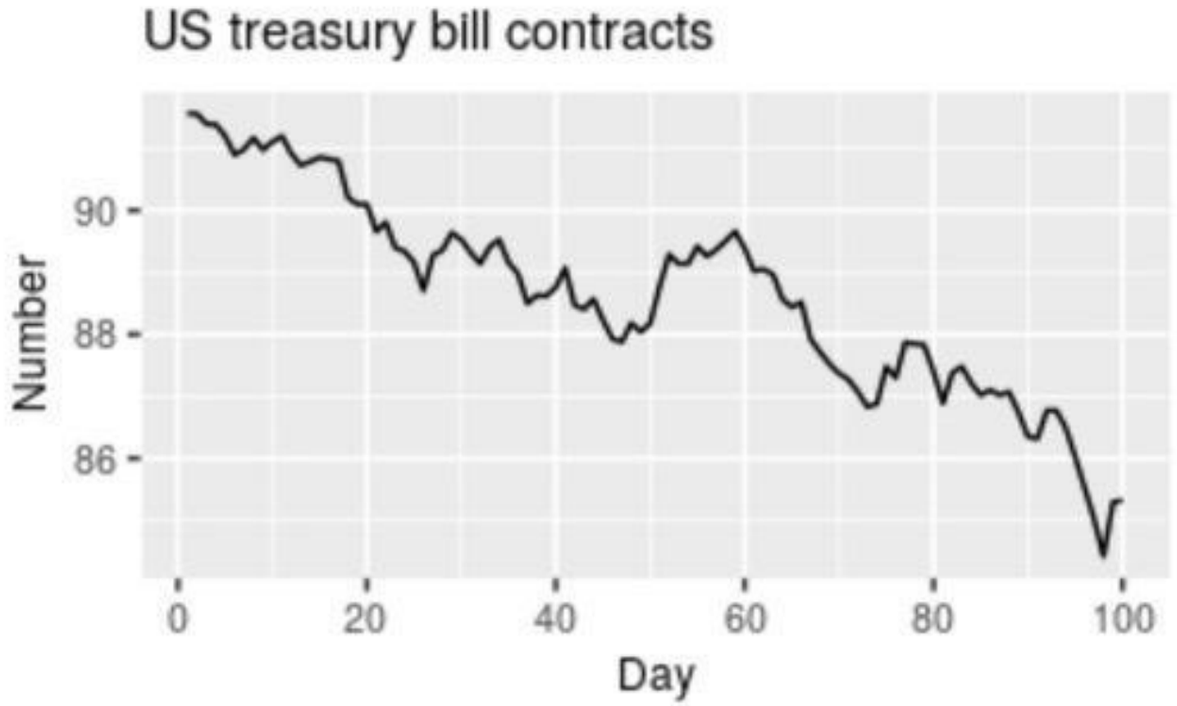


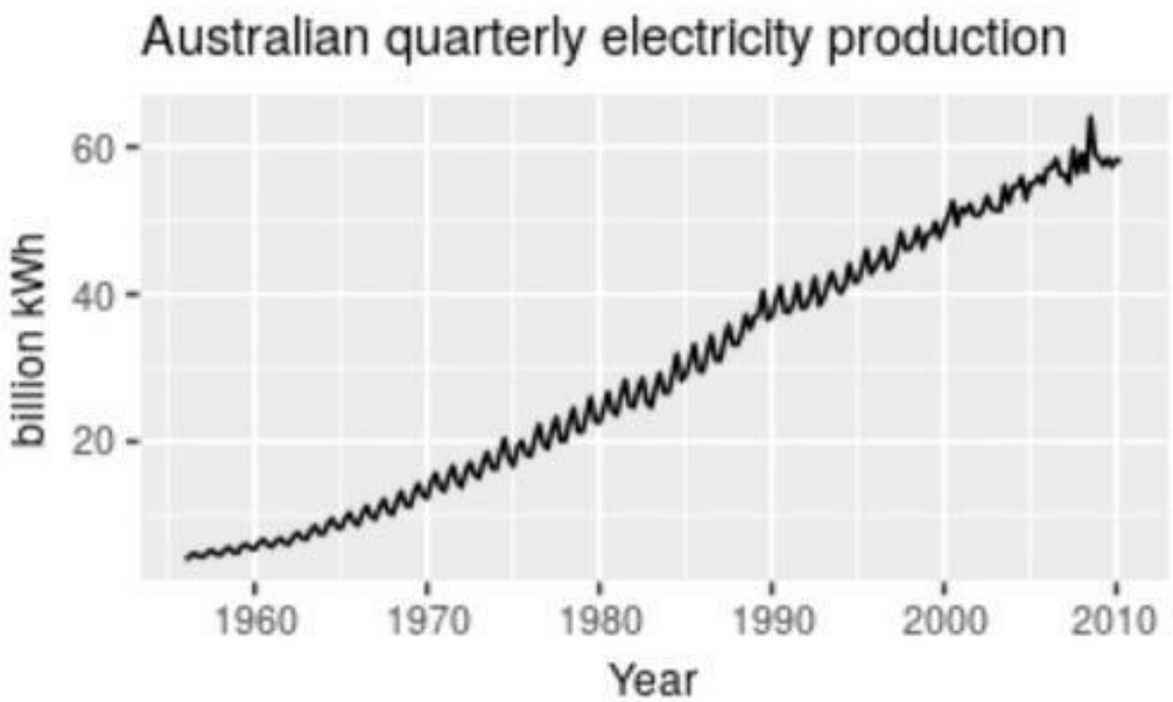
Trend?

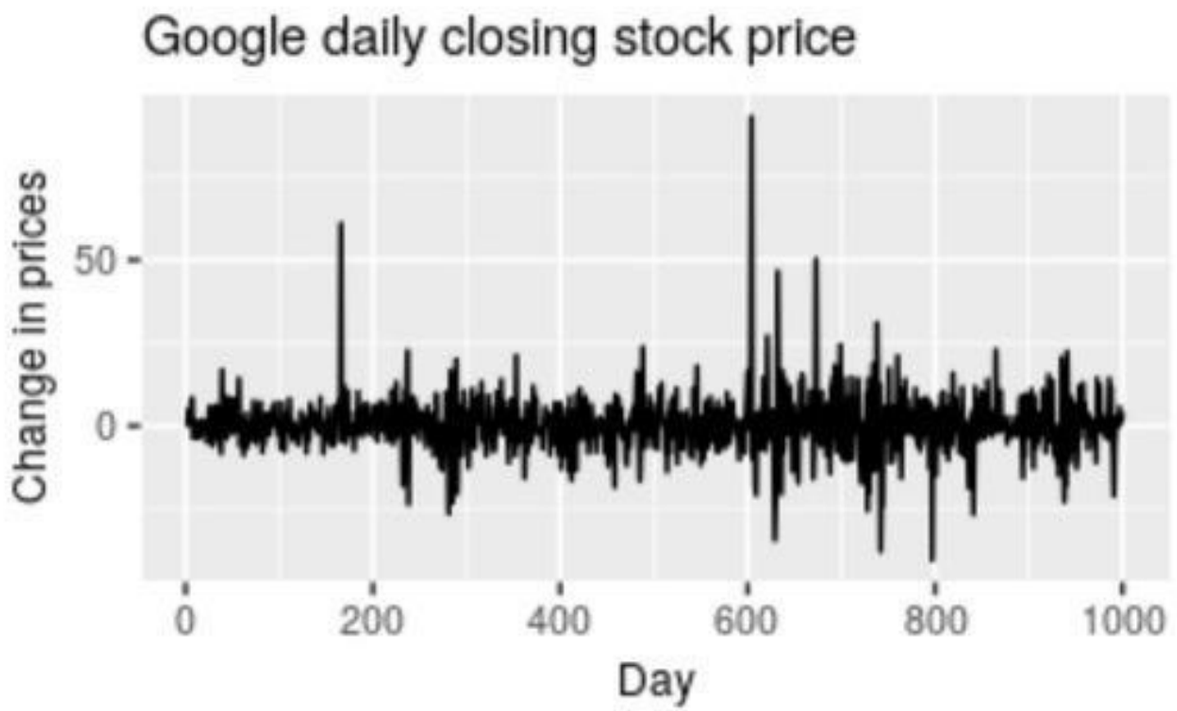
Seasonality?

Cyclic behavior?









Learning to look at Time-Series

00-times_series_visualization.ipynb

What are time series problems

- Classification (e.g. brain computer interfaces, diagnosis from physiological signals like ECG)
- Regression (e.g. brain age prediction from EEG)
- Forecasting (e.g. sales & energy consumption, online user traffic, weather forecast, stock market)
- Anomaly/Event detection (e.g. factory monitoring)
- Segmentation / change point detection (e.g. detecting changes on user demand)
- Survival analysis (churn, attrition, …)

Focus on signal classification

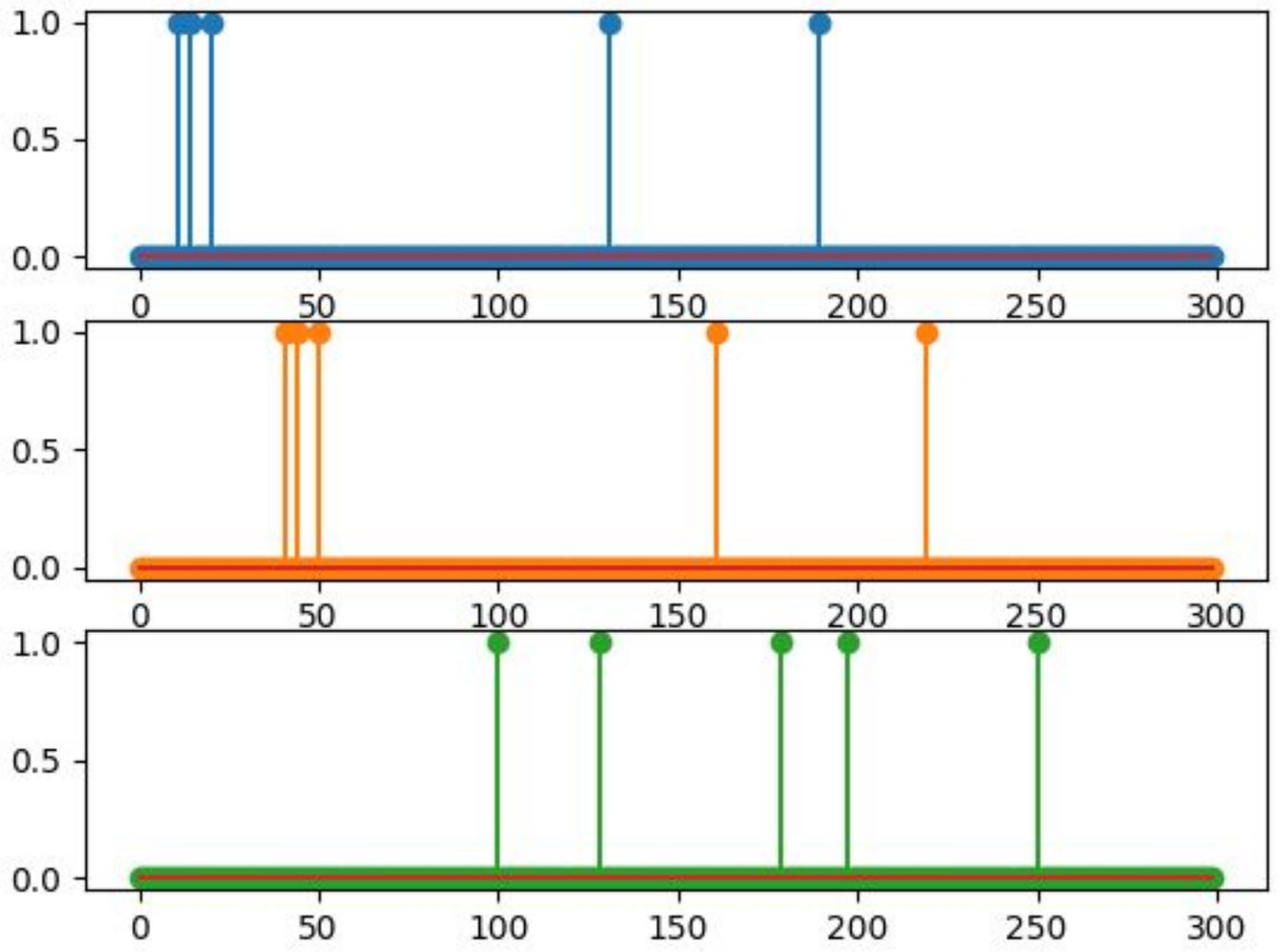
- We have N signals $X^{(i)}$ and we want to predict $y^{(i)}$.
- Consider the full signal to make the prediction.

Problems

- For features, there is a notion of distance between features: we need to account for this.
- Data are "functional data": dedicated metrics, notion of scale, spectral content, patterns, different lengths/scales
- Generic APIs (like scikit-learn) are much harder to design
- The ecosystem is a lot more scattered / messy…

On the notion of alignment

Which signal is closer to the blue one?



They are at the same Euclidean distance!

Moreover, each $X^{(i)}$ can have its own domain Ω_i, with potentially different sizes.

On the notion of alignment

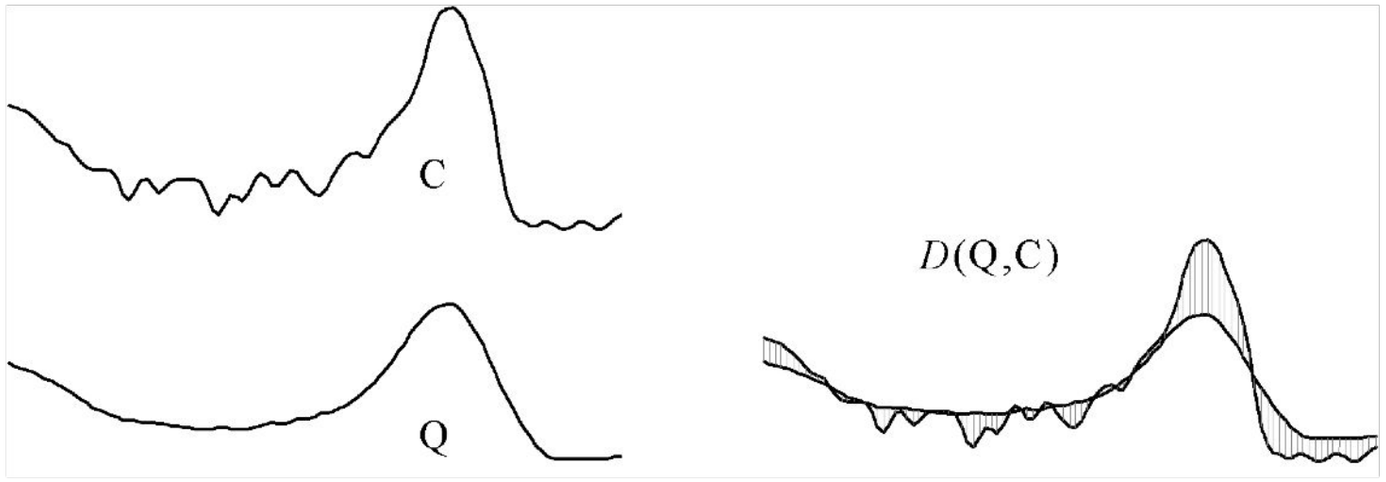
To cope with this, there are several solutions:

- Extract features invariant to alignment
  - Global features: mean, variance, frequencies, …
  - Features based on convolutions
    e.g. w/ deep learning.
- Use metrics that adapt to unalign signals

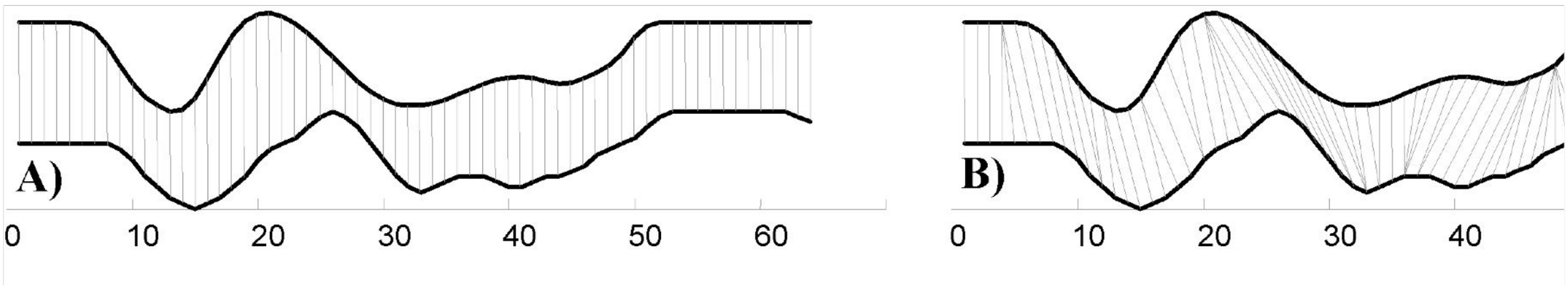
Metrics on time series

- Euclidean distance

$$D(x^1, x^2) = \sqrt{\sum_t (x_t^1 - x_t^2)^2}$$



- Dynamic Time Warping (DTW)



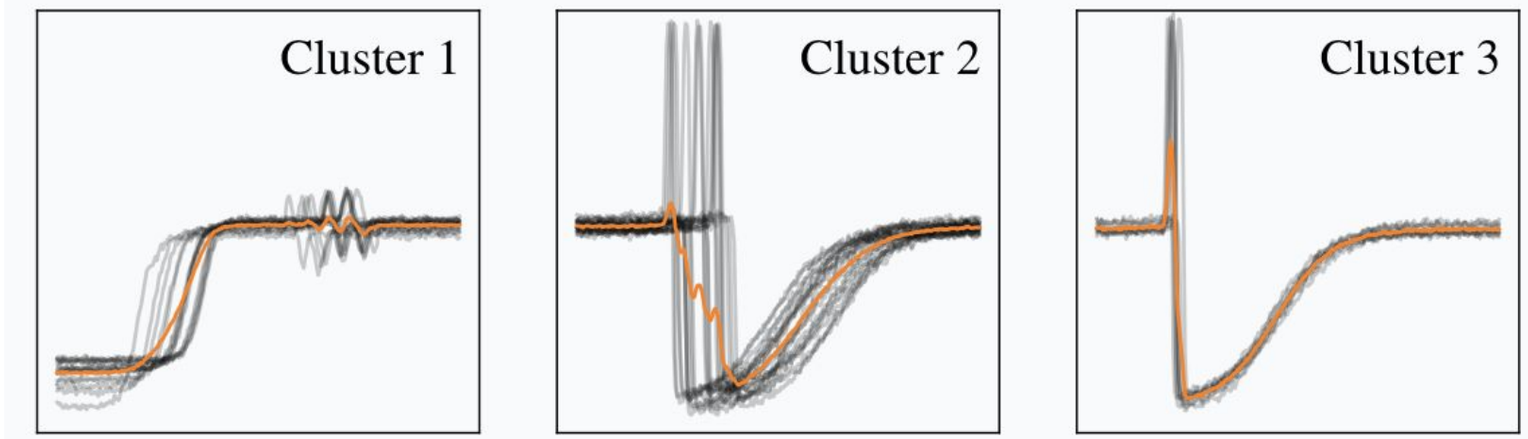
Metrics/distances/similarities are everywhere in ML:
K-means, K-nearest neighbors, kernels / SVMs, etc.

Clustering time series

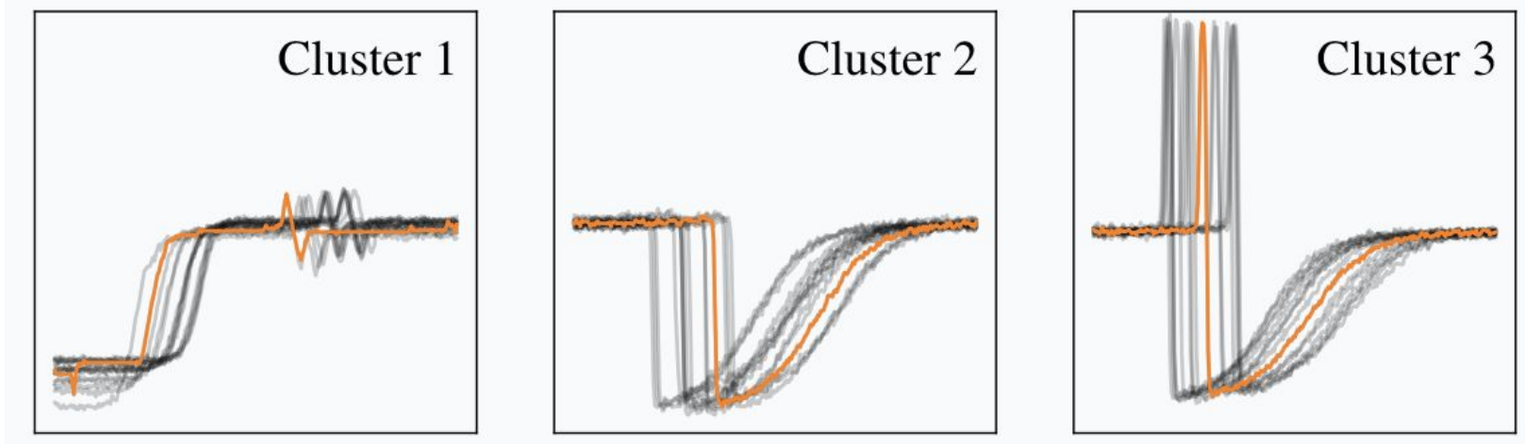
Dataset



K-means result with L2 distance



K-means result with DTW distance



Source:
https://rtavenar.github.io/blog/dtw.html

What are open-source packages for time-series in Python?

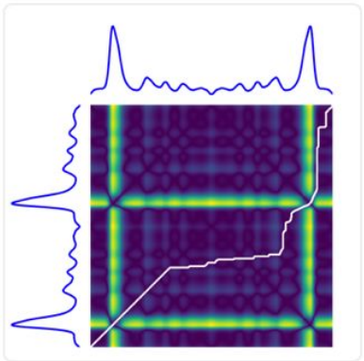
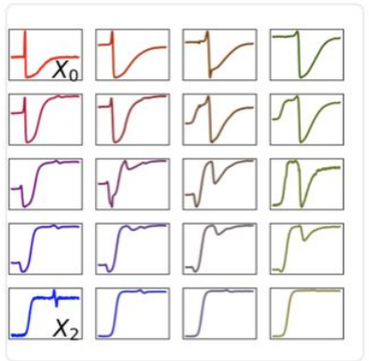
Classifications

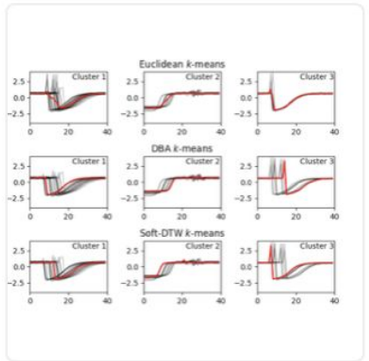
tslearn.readthedocs.io

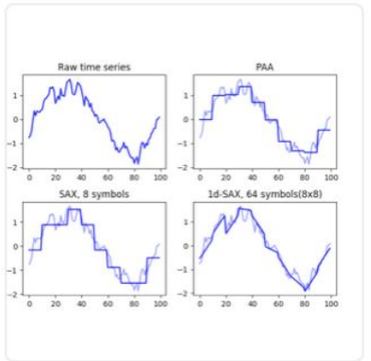
tslearn.readthedocs.io

tslearn 0.5.2 Quick Start User Guide API Examples Citing tslearn Code on GitHub Search

tslearn's documentation





`tslearn` is a Python package that provides machine learning tools for the analysis of time series. This package builds on (and hence depends on) `scikit-learn`, `numpy` and `scipy` libraries.

This documentation contains a quick-start guide (including installation procedure and basic usage of the toolkit), a complete API Reference, as well as a gallery of examples.

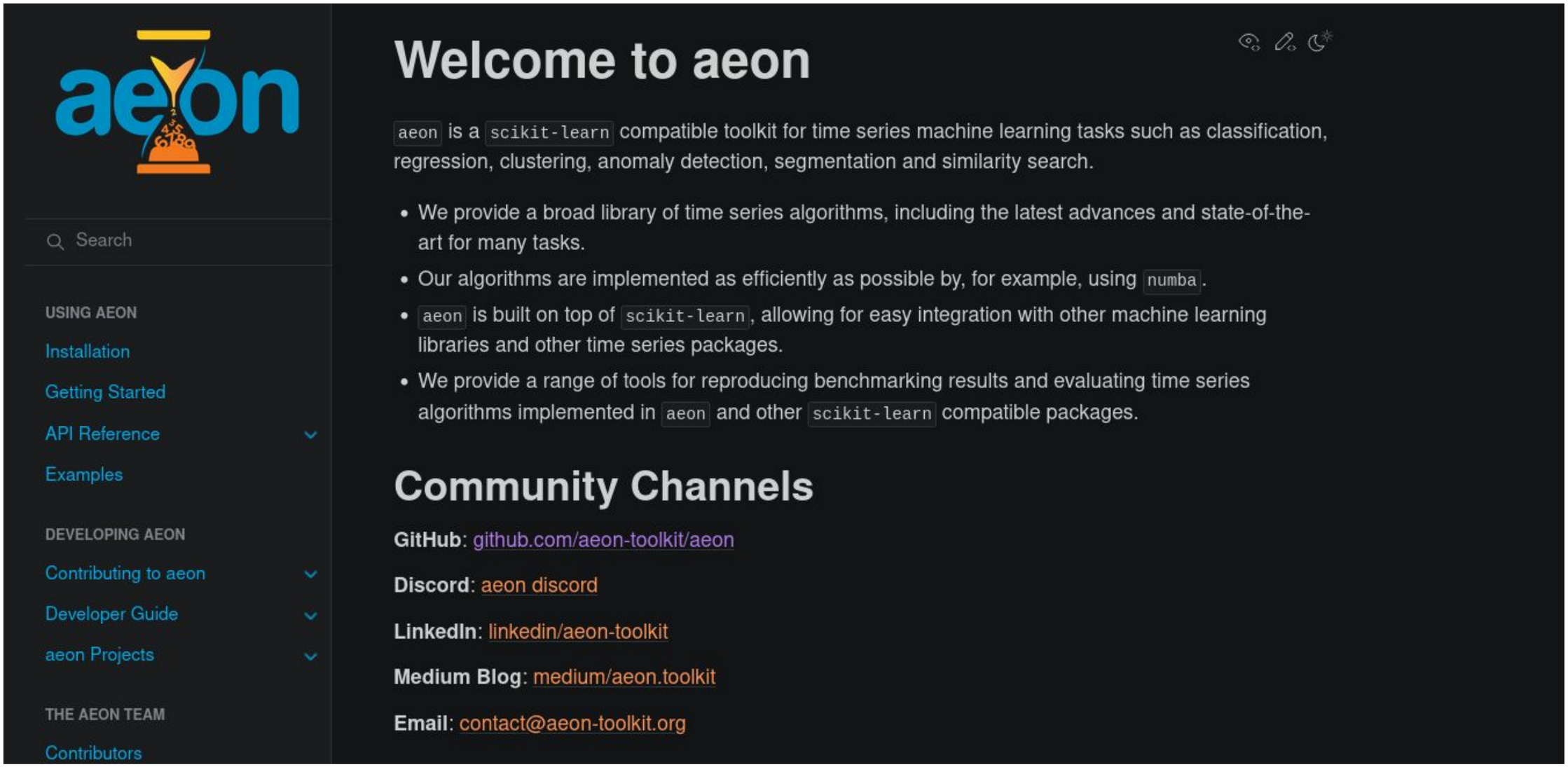
Finally, if you use `tslearn` in a scientific publication, we would appreciate citations.

© Copyright 2017, Romain Tavenard.
Created using Sphinx 1.8.5.

Open "https://tslearn.readthedocs.io/en/stable/auto_examples/clustering/plot_baryce

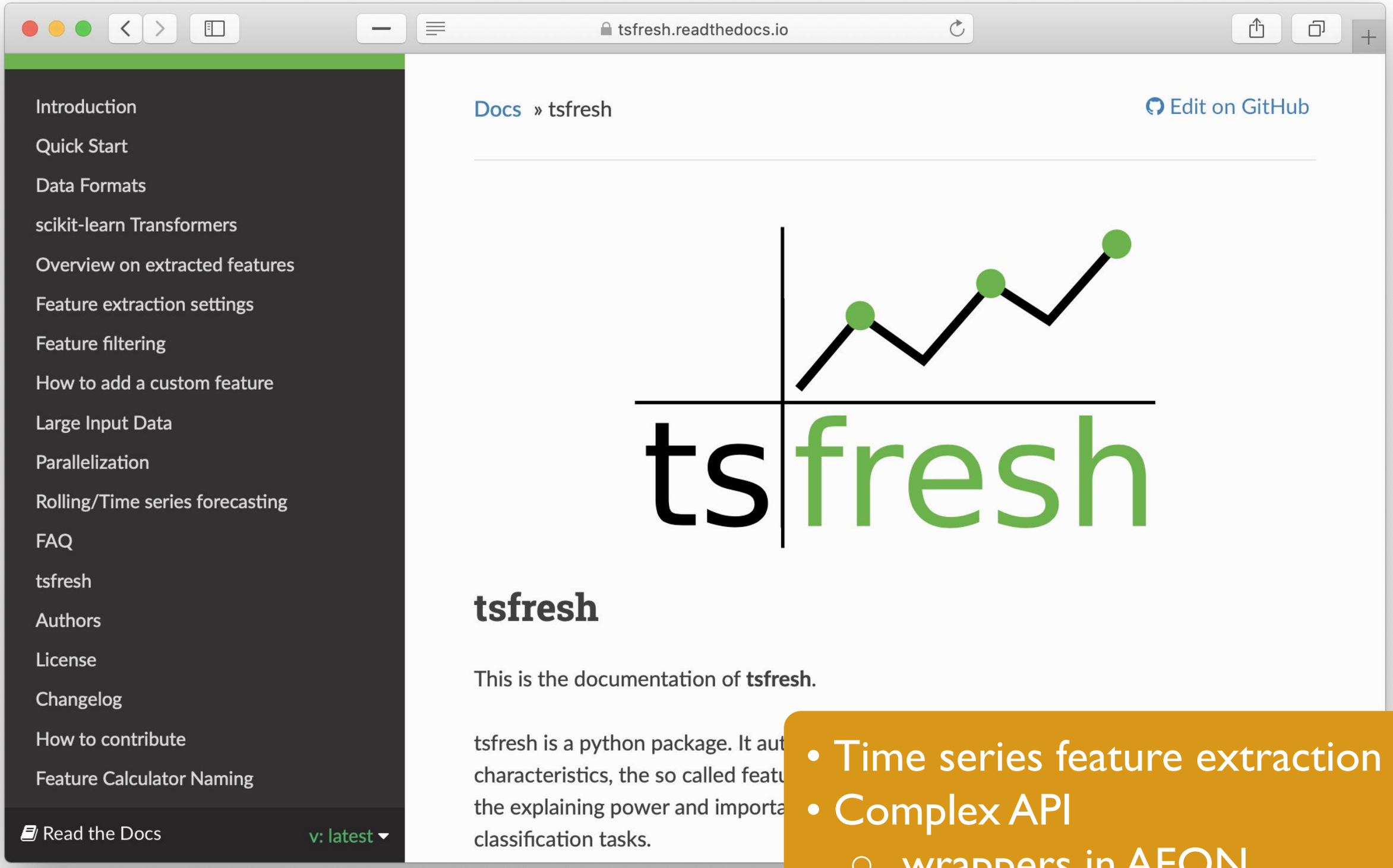
- Distances between time series
- Clustering
- Distance based learning (kNN, kernel based methods)

aeon-toolkit.org



- Classification/Regression/Clustering
- Forecasting
- Scikit-learn compatible API

tsfresh.readthedocs.io/en/latest/



- Time series feature extraction
- Complex API
  - wrappers in AEON

Classifying Time-Series

01-classifying-time-series.ipynb

A side note on samples in TS

We have considered simple tasks where we have one label per signal:

$$p(y^i|x^i_{[1,T]})$$

However, classic tasks with signals involve $p(y_t|x_t)$

- Horizon vs the context of the prediction

$$p(y_{[t,t+H]}|x_t) \qquad p(y_t|x_{[t,t-C]})$$

- One can also extract features

$$p(y_t|z_t) \quad \text{where} \quad z_t = f(x_{[1,t]})$$

We'll now focus on forecasting

From Linear Models to auto-regressive (AR) models

$$y = \sum_{j=1}^{p} x_j \theta_j + \theta_0$$

Example: $\text{sales(t)} = \underbrace{\theta_0 + \theta_1 t}_{\text{trend}} + \underbrace{\theta_2 \cos\left(\frac{2\pi t}{12}\right)}_{\text{seasonal}}$

From Linear Models to auto-regressive (AR) models

$$y = \sum_{j=1}^{p} x_j \theta_j + \theta_0$$

Example: $\text{sales(t)} = \underbrace{\theta_0 + \theta_1 t}_{\text{trend}} + \underbrace{\theta_2 \cos\left(\frac{2\pi t}{12}\right)}_{\text{seasonal}}$

VS

$$x_t = \sum_{k=1}^{p} x_{t-k} \theta_k + \theta_0$$

Example: $\text{sales(t)} = \theta_0 + \underbrace{\theta_1 \text{sales(t} - 1)}_{\text{Previous values}} + \underbrace{\theta_2 \text{sales(t} - 2)}_{\text{Previous values}}$

Recurrent vs. non-recurrent model

$$x_t = f_\theta(t) + g_{\theta'}(z_t)$$

fct. of t — covariates

- Non-recurrent model
- Prediction can be done together for all time points

VS.

$$x_t = f_\theta(x_{t-1}, \ldots, x_{t-p}) + g_{\theta'}(z_t)$$

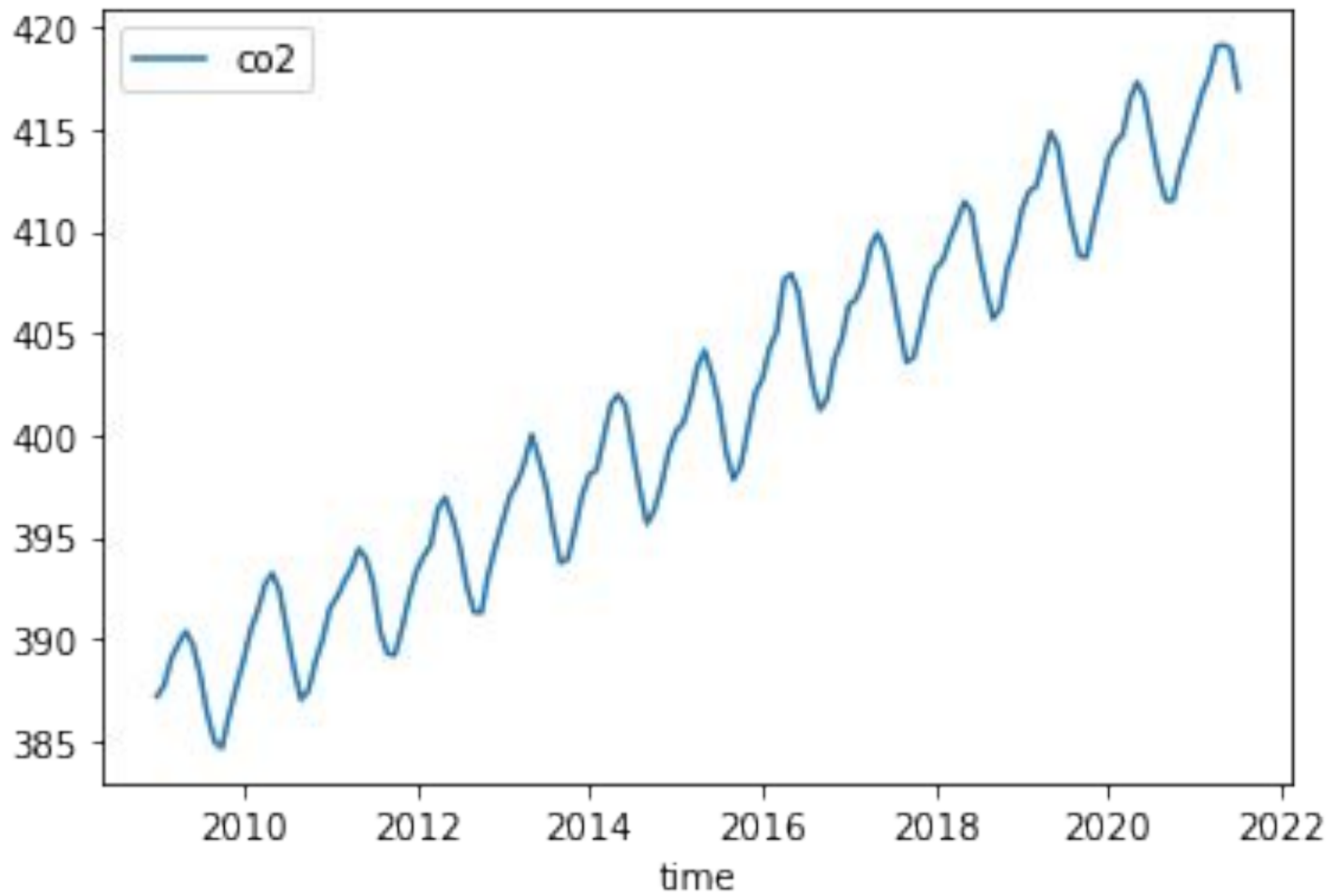
fct. of delayed features — covariates

- Recurrent model
- Prediction is done time-by-time
- Prediction is reinjected into the model

FORECASTING WITH A LINEAR MODEL

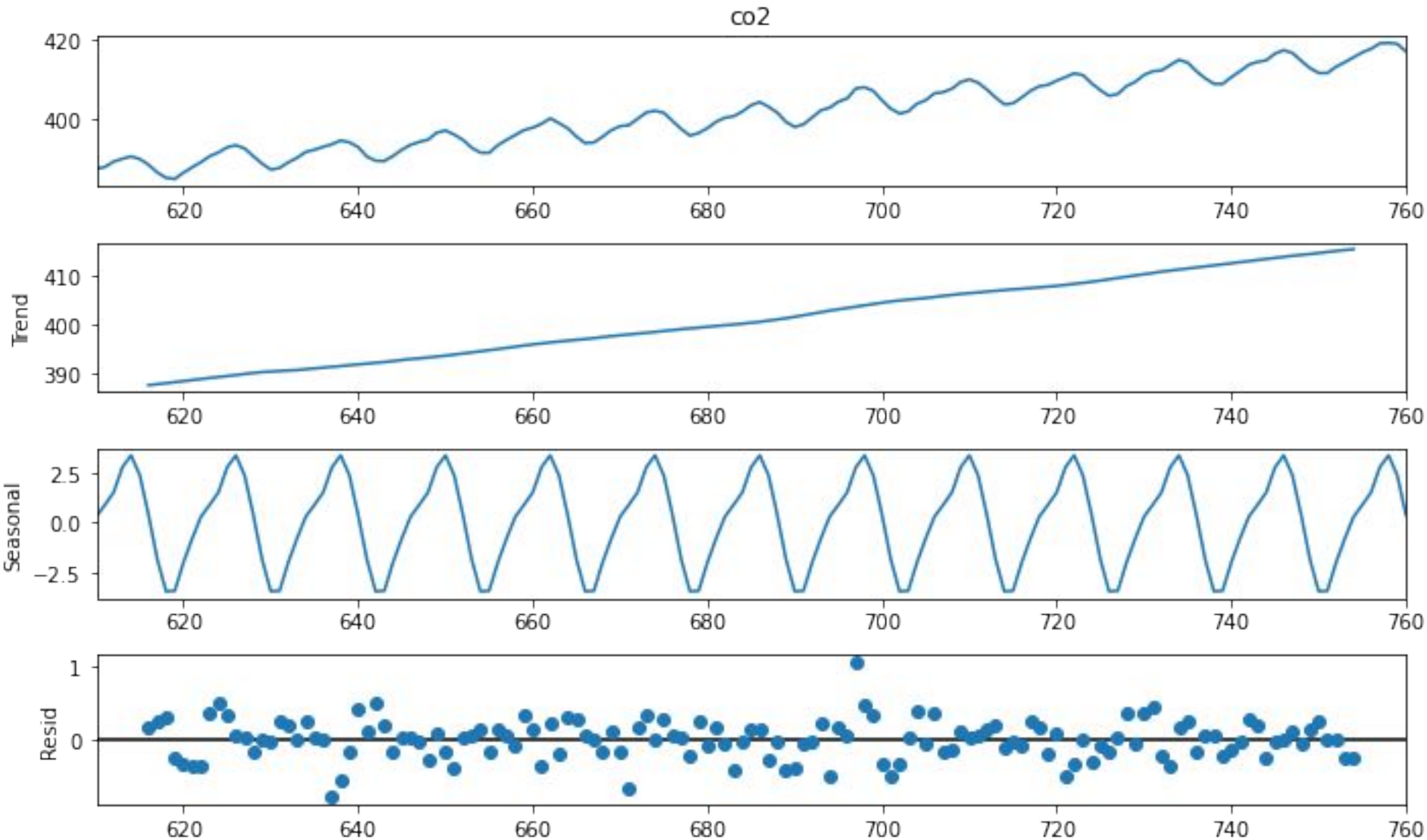
A non-recurrent model:

$$x_t = \theta_0 + \theta_1 t + \theta_2 \cos\left(\frac{2\pi t}{12}\right) + \theta_3 \sin\left(\frac{2\pi t}{12}\right) + \epsilon$$



FORECASTING WITH A LINEAR MODEL

data = trend + seasonal + noise



See: `from statsmodels.tsa.seasonal import seasonal_decompose`

FORECASTING WITH A LINEAR MODEL

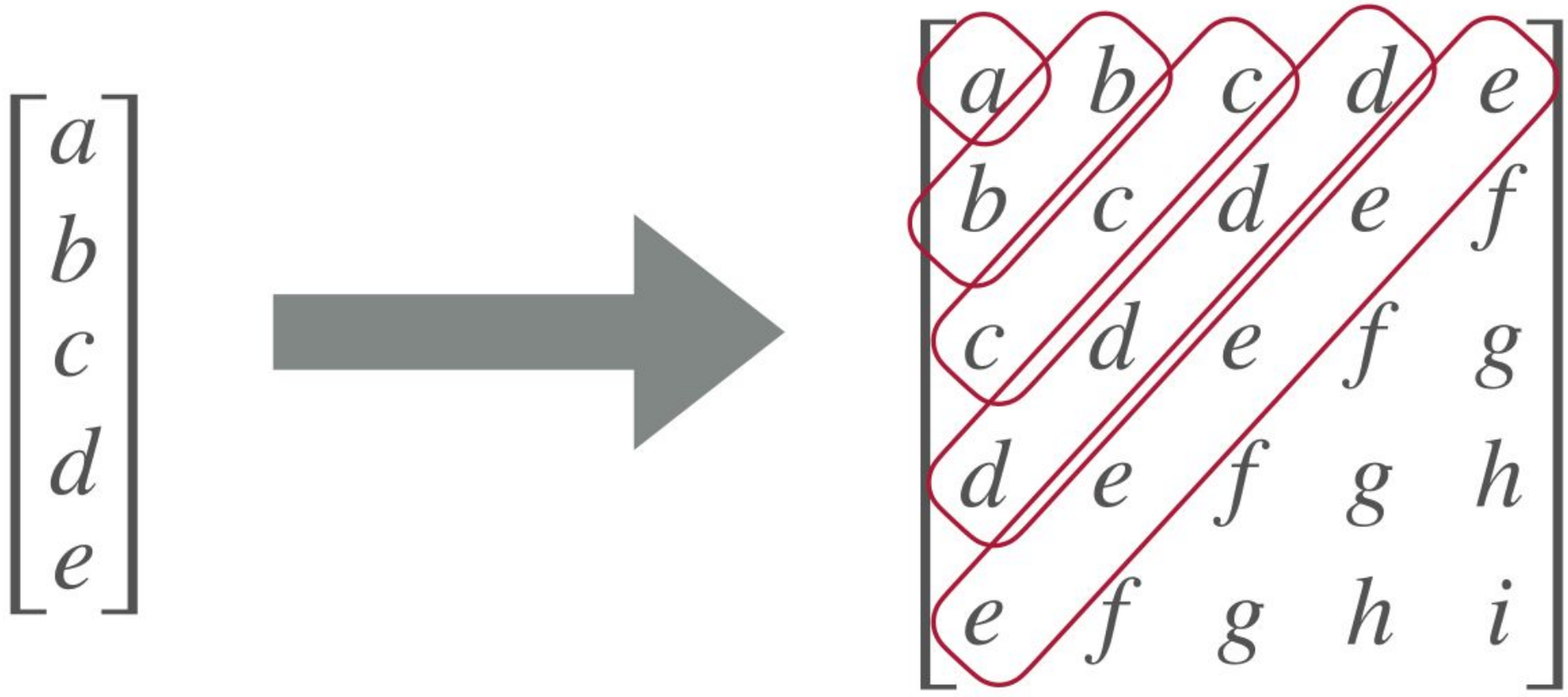
Recurrent model: $x_t = \sum_{k=1}^{p} x_{t-k}\theta_k + \theta_0$ AR(p)

```
See: from statsmodels.tsa.ar_model import AutoReg
```

FORECASTING WITH A LINEAR MODEL

Recurrent model: $x_t = \sum_{k=1}^{p} x_{t-k}\theta_k + \theta_0$ AR(p)

Hankel Matrix a.k.a. "Temporal embedding"



See: `from statsmodels.tsa.ar_model import AutoReg`

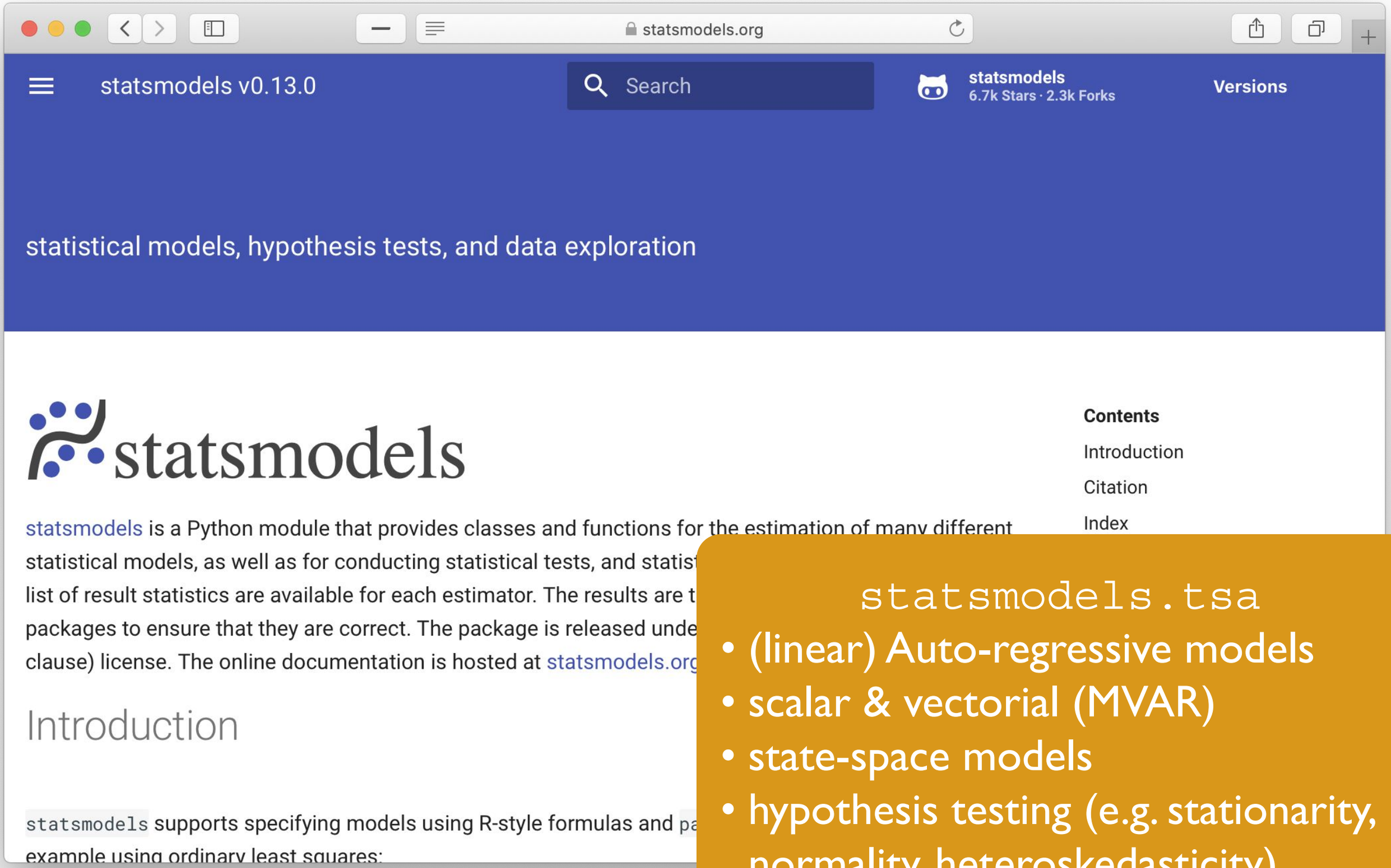
Linear forecasting with auto-regressive (AR) models

02-linear_forecasting_AR.ipynb

What are open-source packages for time-series in Python?

Forecasting

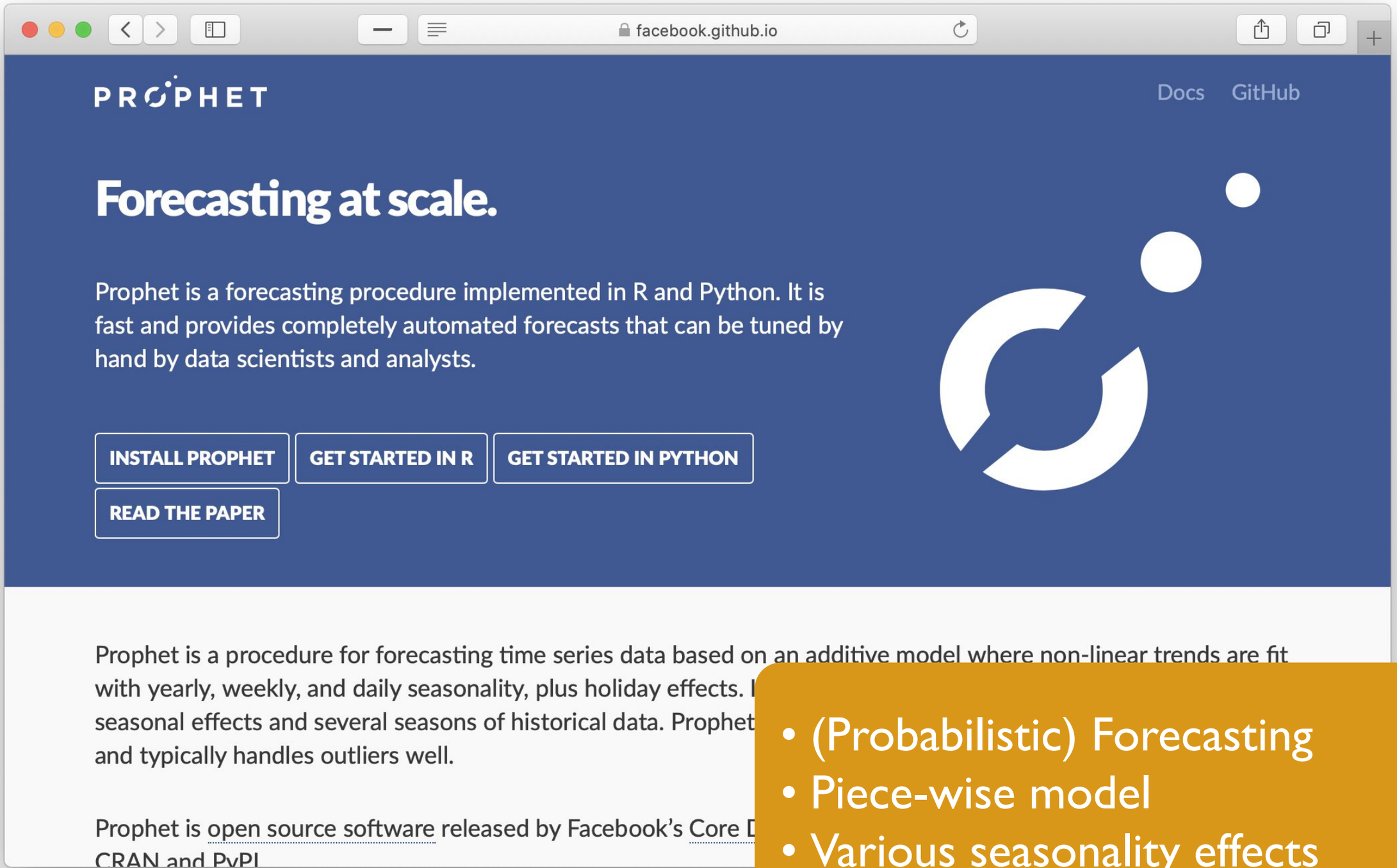
https://www.statsmodels.org/stable/index.html



`statsmodels.tsa`

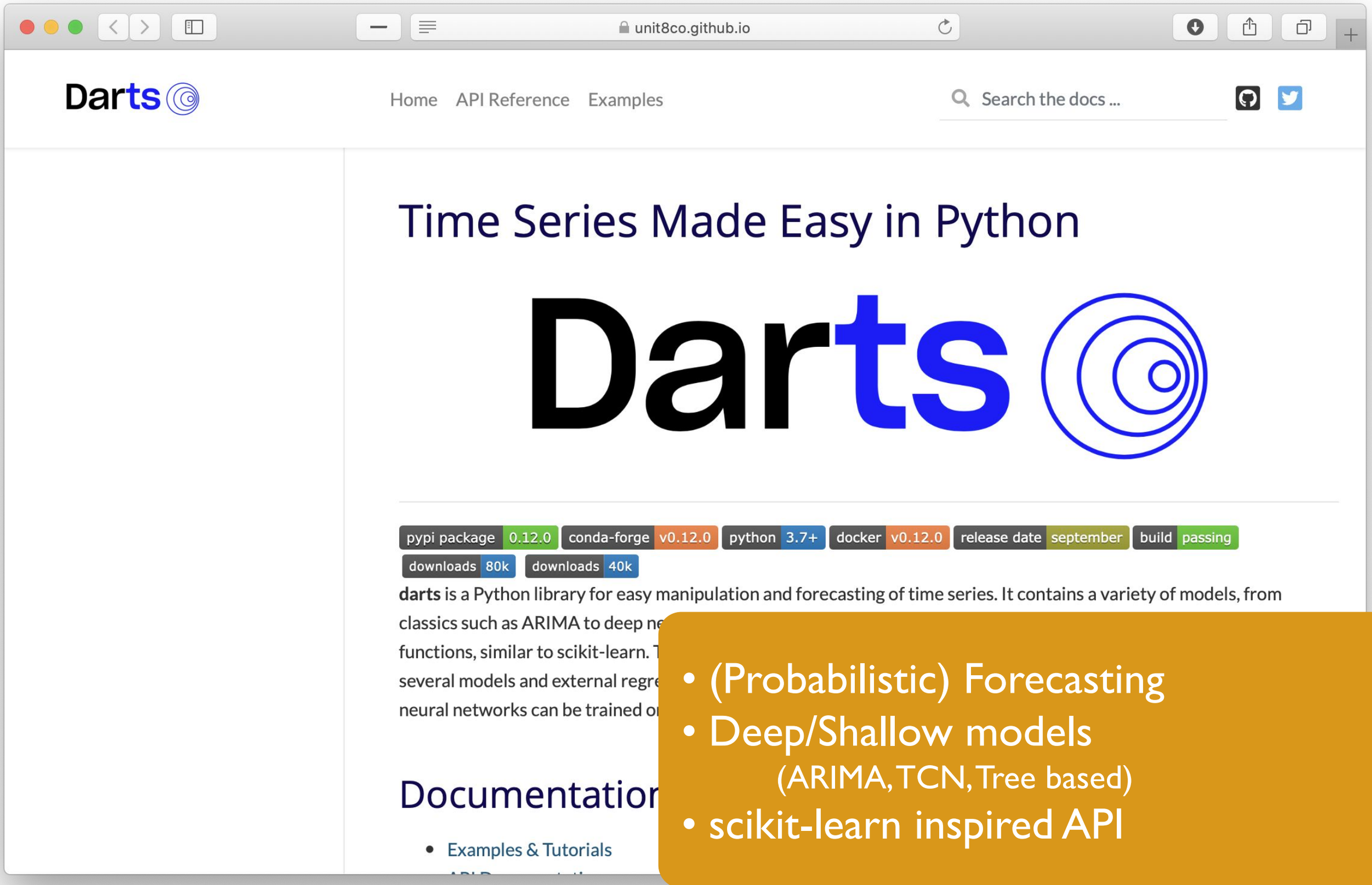
- (linear) Auto-regressive models
- scalar & vectorial (MVAR)
- state-space models
- hypothesis testing (e.g. stationarity, normality, heteroskedasticity)
- fit summary with (p-values)

https://facebook.github.io/prophet/

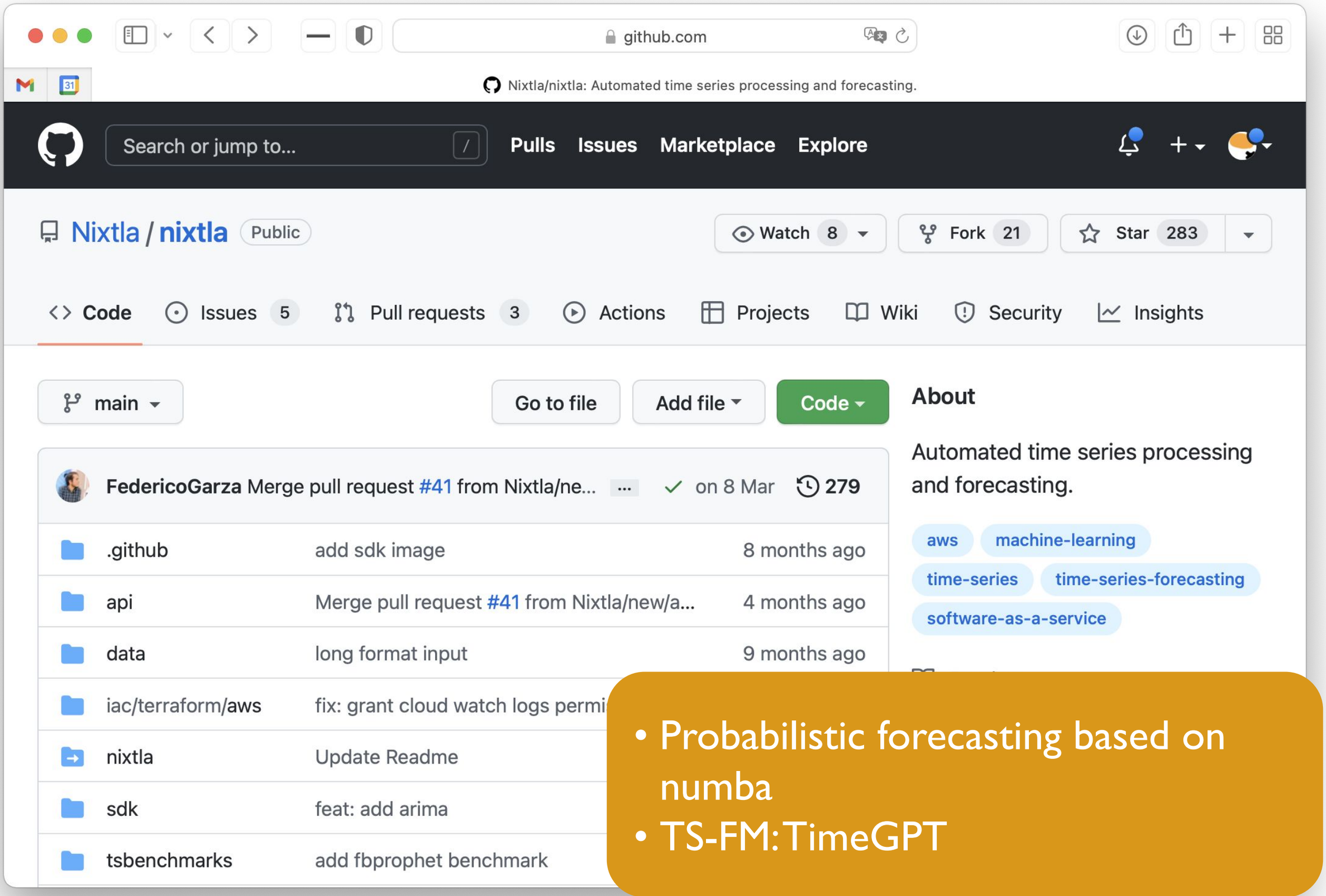


- (Probabilistic) Forecasting
- Piece-wise model
- Various seasonality effects
- Take care of holidays covariates
- missing-data support

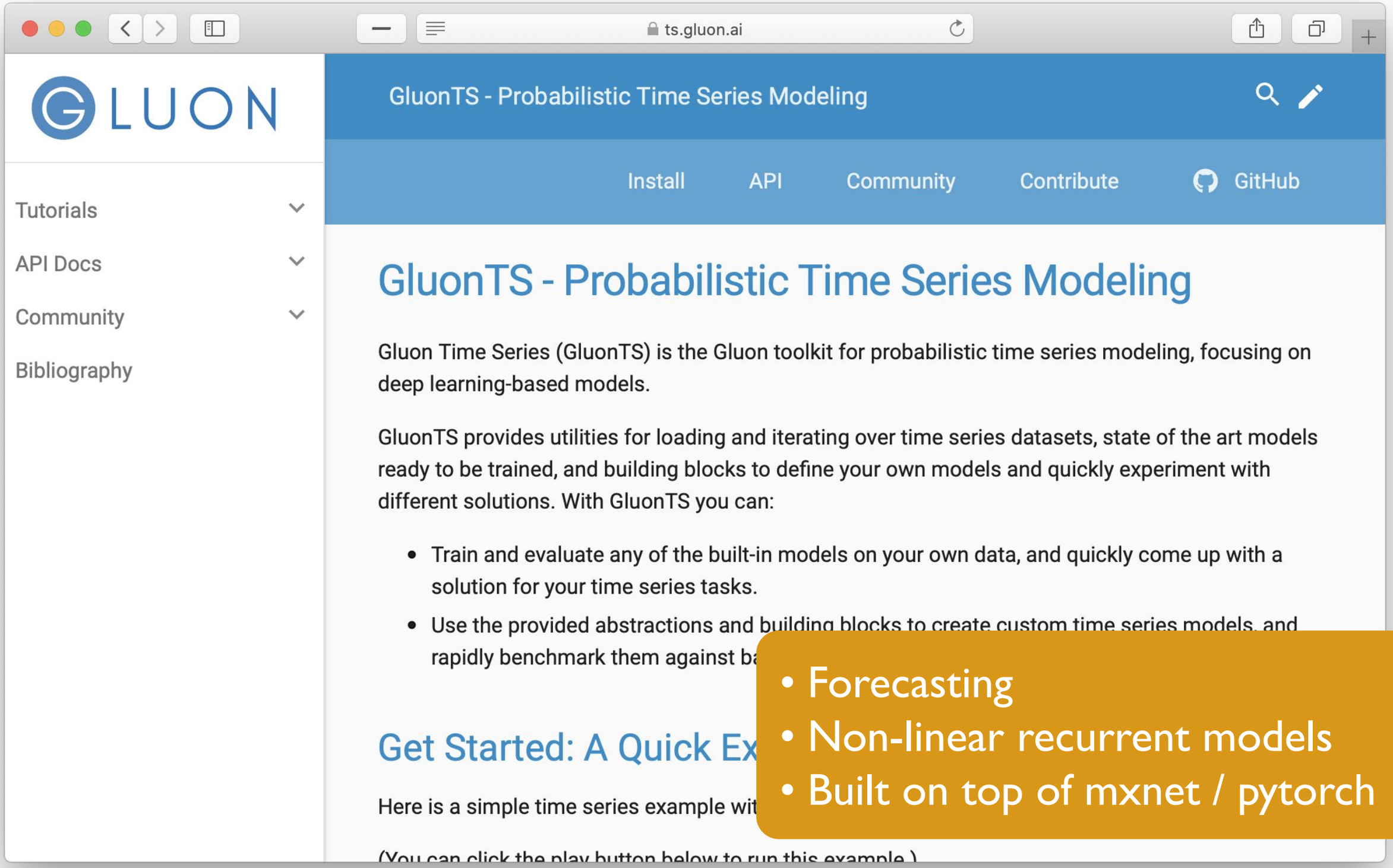
https://unit8co.github.io/darts/



https://github.com/Nixtla/nixtla

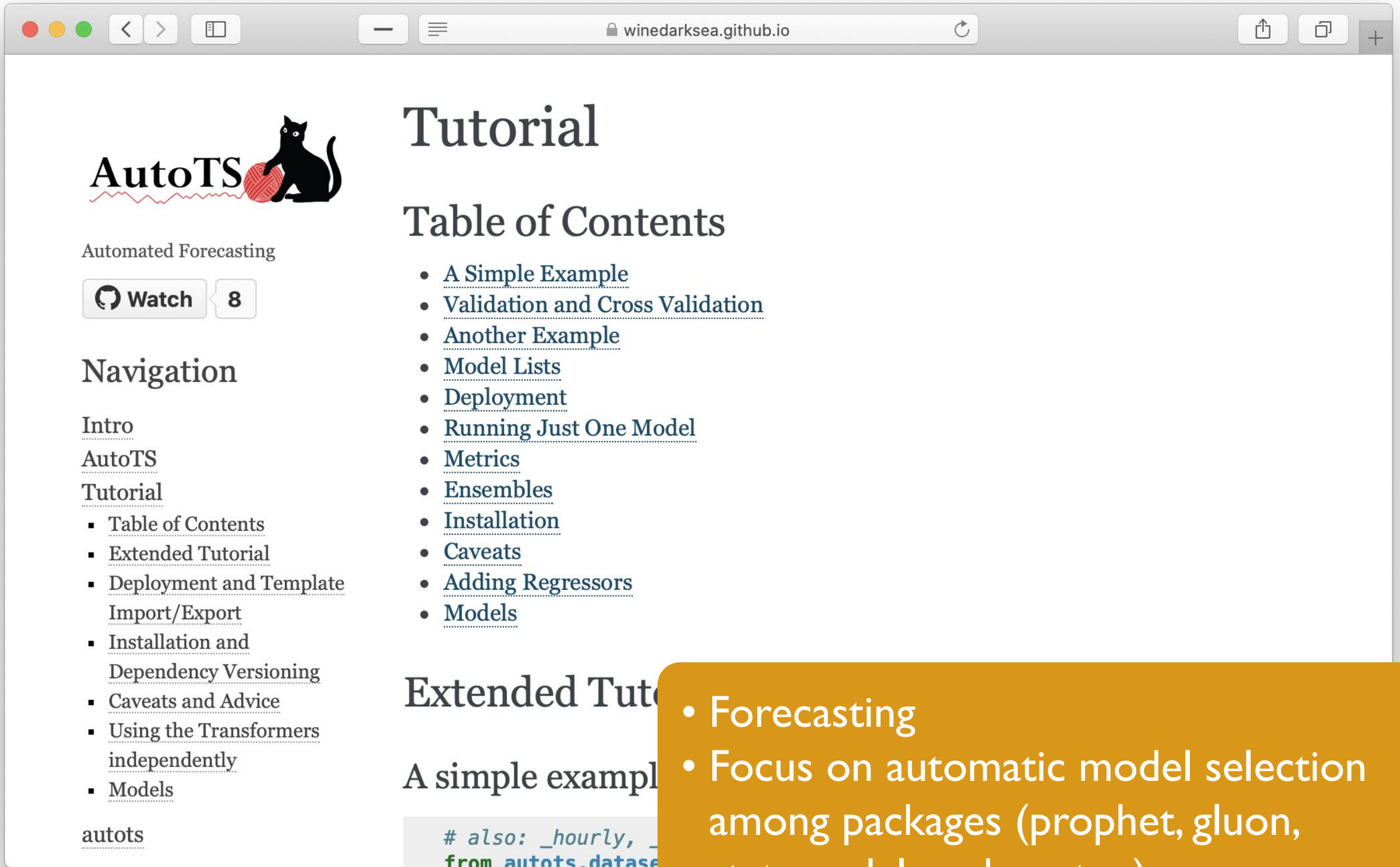


https://ts.gluon.ai



https://arxiv.org/abs/1906.05264

https://winedarksea.github.io/AutoTS



- Forecasting
- Focus on automatic model selection among packages (prophet, gluon, statsmodels, xgboost…)

What are open-source packages for time-series in Python?

Other tasks

https://centre-borelli.github.io/ruptures-docs/

centre-borelli.github.io

ruptures | Search | deepcharles/ruptures

Welcome to ruptures

Maintained? yes | build passing | python 3.6 | 3.7 | 3.8 | 3.9 | pypi package 1.1.5 | conda-forge v1.1.5 | docs passing | license BSD-2-Clause | downloads 3M | code style black | launch binder | codecov 97%

`ruptures` is a Python library for off-line change point detection. This package provides methods for the analysis and segmentation of non-stationary signals. Implemented algorithms include exact and approximate detection for various parametric and non-parametric models. `ruptures` focuses on ease of use by providing a well-documented and consistent interface. In addition, thanks to its modular structure, different algorithms and models can be connected and extended within this package.

How to cite. If you use `ruptures` in a scientific publication, we would appreciate citations to the following paper:

- C. Truong, L. Oudre, N. Vayatis. Selective review of offline change point detection methods. *Signal Processing*, 167:107299, 2020. [journal] [pdf]

Basic usage

(Please refer to the documentation for more advanced use.)

Table of contents

Basic usage
General information
Contact
Important links
Dependencies and install
Changelog
Thanks to all our contributors
License

- Change point detection

https://stumpy.readthedocs.io

STUMPY documentation

pypi v1.12.0 | Anaconda.org 1.12.0 | downloads/month 183k | license 3-clause BSD License | Tests passing | codecov 100%

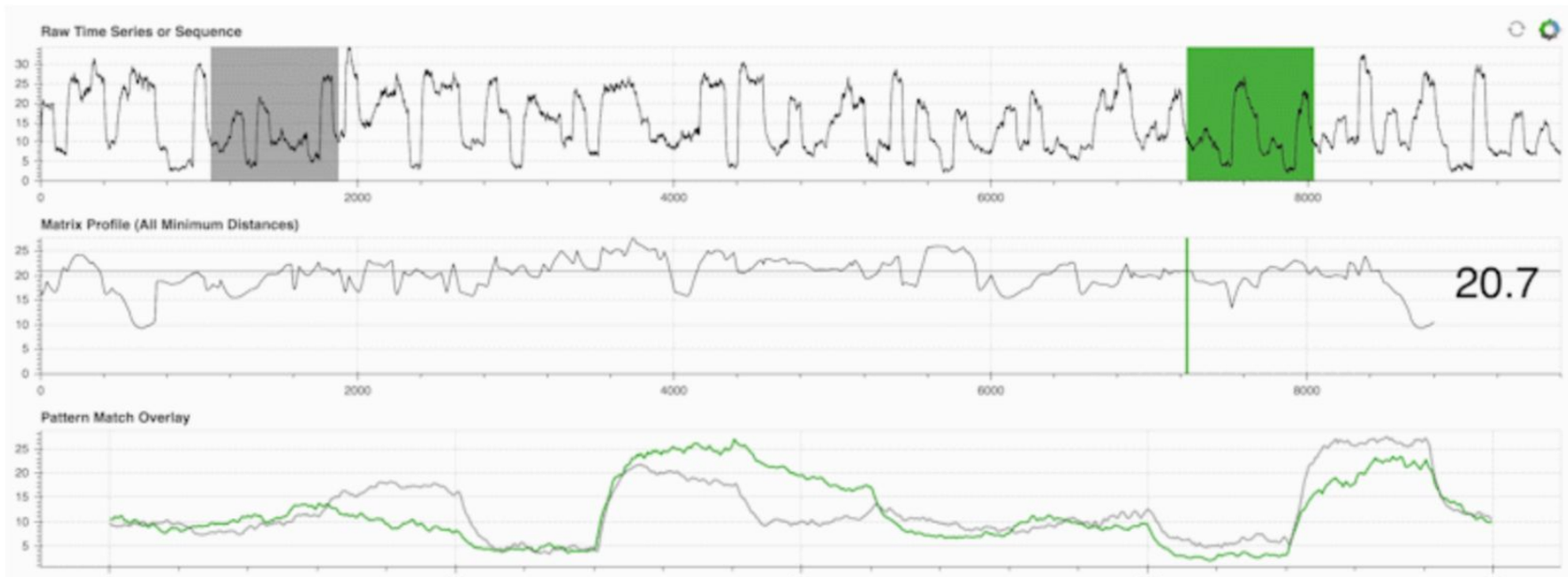
docs passing | launch binder | JOSS 10.21105/joss.01504 | NumFOCUS Affiliated Project | license scan passing

stumpy

- Exploratory Data Analysis

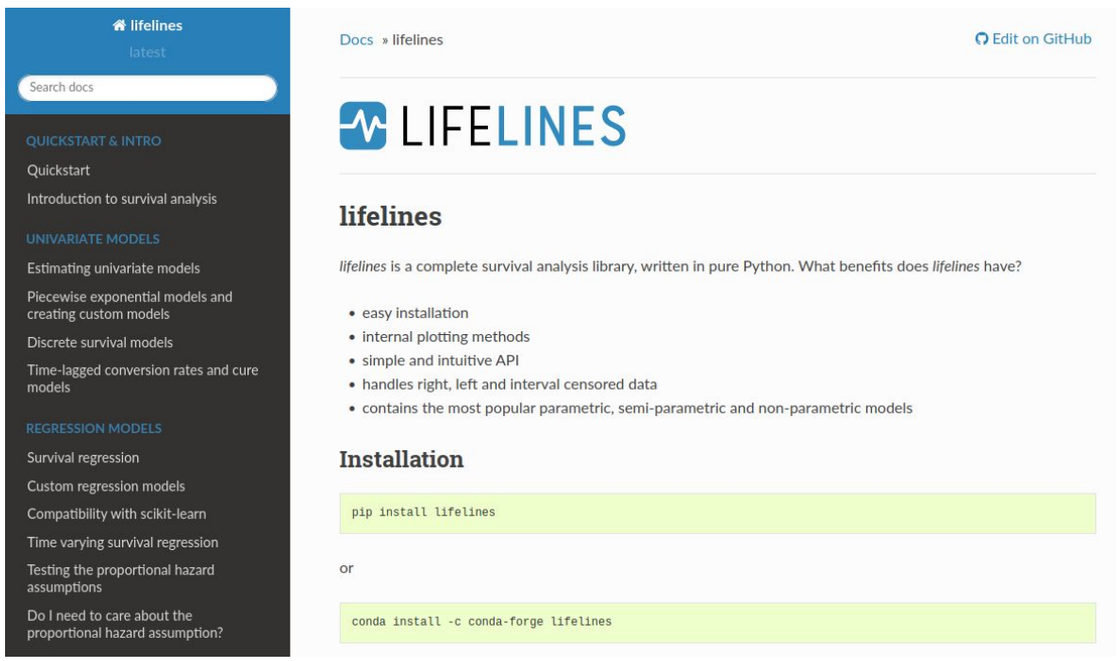
STUMPY

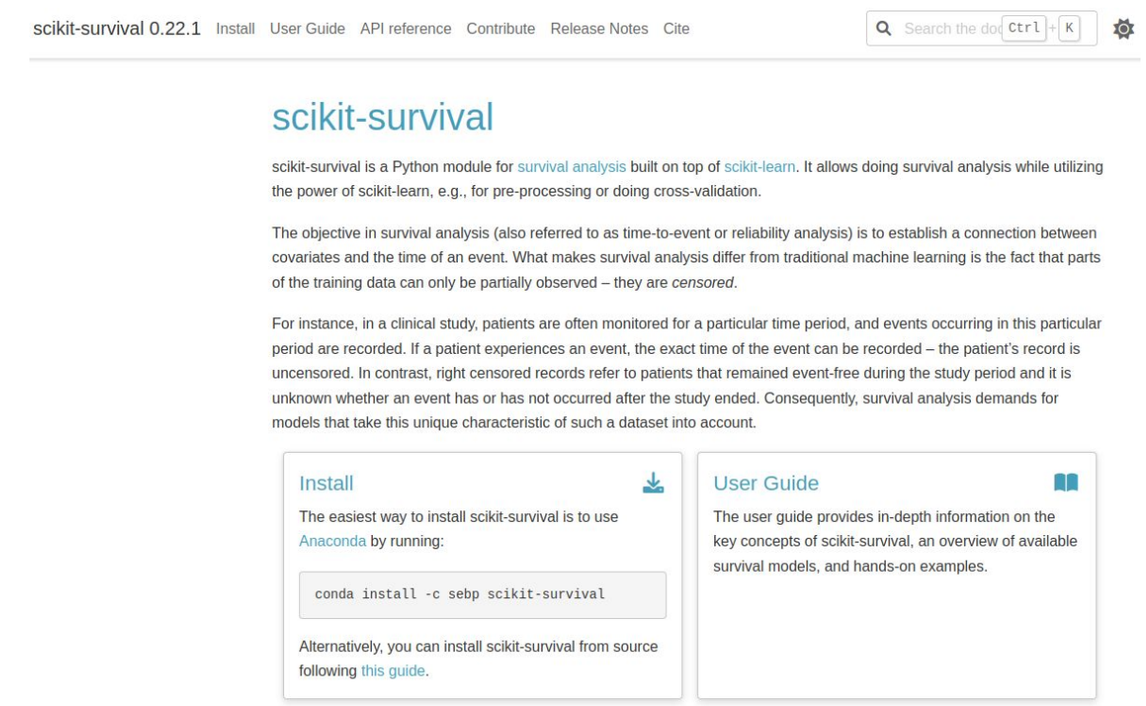
STUMPY is a powerful and scalable Python library that efficiently computes something called the matrix profile, which is just an academic way of saying “for every (green) subsequence within your time series, automatically identify its corresponding nearest-neighbor (grey)”:



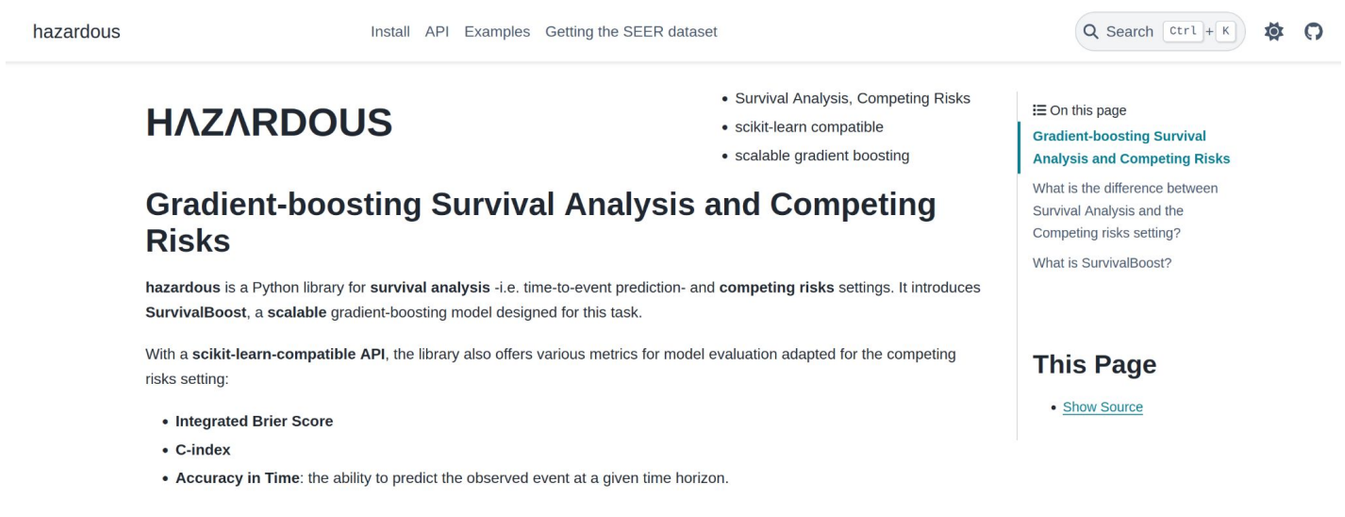
Survival Analysis

https://lifelines.readthedocs.io





https://scikit-survival.readthedocs.io



https://soda-inria.github.io/hazardous/

A first case study

Forecasting Atmospheric Carbon Dioxide

03-forecasting_C02.ipynb

A second case study

Forecasting Bike usage

04-forecasting_case_study.ipynb

An example beyond forecasting

—

Clustering from time series

05-clustering_time_series.ipynb

Questions?